

THE UNIVERSITY OF ZAMBIA

SCHOOL OF NATURAL AND APPLIED SCIENCES

DEPARTMENT OF COMPUTING AND INFORMATICS

FINAL YEAR PROJECT REPORT

ACADEMIC YEAR 2026

Tranz-Algo: Explainable Landed-Cost Inventory Orchestration with Dual Cost Modes for Import-Dependent SMEs

A Final Year Project Report Submitted to the

Department of Computing and Informatics

in partial fulfilment of the requirements for the award of

Bachelor of Science in Computer Science

Student Name: Nixon Shibuchi

Computer Number: 2021437361

Group project with: Musonda Katongo (2020583969)

Supervisor: Mrs. Leena Kumar

Date of Submission: 23rd October 2026

Source code repository: `https://github.com/Nixon-Shibuchi/Tranz-Algo`

Declaration

I declare that this final-year project report is my own work and has been prepared in accordance with the academic requirements of the University of Zambia. All sources of information, ideas, data, software, images, and other material that are not my own have been appropriately acknowledged and referenced.

Where this project was undertaken as a group, I have clearly identified my individual contributions and have not presented the work of other group members as my own. Tranz-Algo was developed with Musonda Katongo. Design and technology decisions were made jointly. My primary role was implementation lead: FastAPI backend, costing engine, most frontend wiring, FX service, evaluation scenarios/oracles, literature lead drafts, and majority of commits. See Appendix A.

Student Name: Nixon Shibuchi

Computer Number: 2021437361

Signature: _____

Date: 23rd October 2026

Approval

This final year project report has been submitted for examination with the approval of the project supervisor.

Supervisor Name: Mrs. Leena Kumar

Signature: _____

Date: _____

Departmental/Examiner Approval (if applicable): _____

Abstract

Retailers who import usually have the supplier invoice and the clearing sheet, and they still struggle to say what one item in the batch actually cost. Freight, duty and VAT are billed on the whole shipment, not on each item. The papers often mix a foreign unit price with amounts already in kwacha, and the rate can move after the price was first set.

Tranz-Algo is the web system I built for that job. It turns the shipment papers into a landed cost per item, a suggested retail price, and a record of how the price was worked out. There are two ways to enter cost, Auto FX and Already-Local, and the shared charges can be split by quantity or by weight with both results shown together. True Cost then suggests a price and warns if it falls under a safety floor. Stock is updated from the batch. Record sale only reduces stock. It is not a full till.

The build uses FastAPI, a React (Vite) admin screen, and PostgreSQL, with SQLite if no Postgres address is set. Exchange rates come from the free Frankfurter service when the user asks for one, with a saved rate and a box to type a rate if the service is down. I checked two demo shipments against spreadsheets, an Already-Local grocery batch and an Auto FX phone-parts batch, and the figures stayed within K0.01. The point of the project is import-batch costing that can be explained when the papers are mixed. It is not another stock list and it is not an ERP.

This individual report was prepared by Nixon Shibuchi for the group project developed in collaboration with Musonda Katongo, and it is submitted independently for examination. The shared implementation and source code remain publicly accessible on GitHub at `https://github.com/Nixon-Shibuchi/Tranz-Algo`.

Keywords: landed cost; dual cost modes; import SME; allocation; True-Cost Ledger; FastAPI; React

Acknowledgements

I thank my supervisor, Mrs. Leena Kumar, for guidance throughout this final year project. I also thank the Department of Computing and Informatics, University of Zambia, for providing the academic framework and assessment standards that shaped this work. Finally, I acknowledge Musonda Katongo for joint design discussions and complementary implementation effort that produced the working Tranz-Algo MVP.

Table of Contents

Declaration i

Approval ii

Abstract iii

Acknowledgements iv

List Of Abbreviations And Acronyms xi

Note On Group Projects xii

1 Introduction 1

1.1 Background and Context 1

1.2 Problem Statement 1

1.3 Project Aim 2

1.4 Project Objectives 2

1.5 Research Questions 2

1.6 Scope of the Project 2

1.6.1 In Scope 2

1.6.2 Out of Scope 3

1.7 Significance of the Project 3

1.8 Limitations 3

1.9 Organisation of the Report 4

2 Literature Review 5

2.1 Introduction 5
2.2 Key Concepts and Theories 5
2.3 Existing Systems / Related Work 7
2.4 Comparative Analysis 8
2.5 Research / Knowledge Gap 8
2.6 Conceptual Framework 8
2.7 Chapter Summary 9

3 Methodology **10**
3.1 Introduction 10
3.2 Research / Development Approach 10
3.3 Requirements / Data Collection 10
3.4 System Architecture / Research Workflow 11
3.5 Tools and Technologies 12
3.6 Ethical and Legal Considerations 12
3.7 Evaluation Method 12
3.8 Chapter Summary 12

4 System Analysis and Design **13**
4.1 Introduction 13
4.2 Stakeholder and User Analysis 13
4.3 Functional Requirements 13
4.4 Non-Functional Requirements 14
4.5 Use Cases / User Stories 14
4.6 System / Solution Architecture 14
4.7 Database / Data Design 15
4.8 Interface / Interaction Design 16
4.9 Algorithms / Models / Technical Design 20
4.10 Chapter Summary 21

5 Implementation / Development **22**

5.1 Introduction . . . 22
5.2 Development Environment . . . 22
5.3 System Components / Modules . . . 22
5.4 Key Implementation Details . . . 23
5.5 Integration . . . 23
5.6 Challenges and Solutions . . . 23
5.7 Individual Contribution (group projects) . . . 24
5.8 Chapter Summary . . . 24

6 Testing, Results and Evaluation 25

6.1 Introduction . . . 25
6.2 Testing Strategy . . . 25
6.3 Test Cases and Results . . . 26
6.4 Experimental Results . . . 26
6.5 User Evaluation . . . 28
6.6 Analysis of Results . . . 29
6.7 Objective-by-Objective Evaluation . . . 29
6.8 Individual Results . . . 29
6.9 Chapter Summary . . . 30

7 Discussion 31

7.1 Introduction . . . 31
7.2 Interpretation of Findings . . . 31
7.3 Comparison with Existing Work . . . 31
7.4 Contributions of the Project . . . 32
7.5 Limitations and Implications . . . 32
7.6 Chapter Summary . . . 32

8 Conclusion and Recommendations 33

8.1 Introduction . . . 33
8.2 Summary of the Project . . . 33

8.3 Conclusions . . . 33
8.4 Achievement of Objectives . . . 33
8.5 Recommendations . . . 34
8.6 Future Work . . . 34

References **36**

A Individual Contribution Statement **37**

A.1 Project and Group Information . . . 37
A.2 Overall Group Project . . . 37
A.3 Individual Role and Responsibilities . . . 38
A.4 Collaboration and Co-author Work . . . 38

B User Manual / Installation Guide (summary) **39**

B.1 Demonstration sequence . . . 39
B.2 Persistence . . . 40

C Worked Costing Examples **41**

C.1 Shared-charge identity . . . 41
C.2 Demo A, rice line . . . 41
C.3 Demo A by weight, rice line . . . 41
C.4 Demo B, earbud line at rate 20 . . . 42
C.5 Demo B by quantity, earbud line at rate 20 . . . 42
C.6 What a mismatch would mean . . . 42

D Requirements Traceability **43**

List of Figures

2.1 Dual cost-entry modes conceptual framework 9

3.1 System architecture used in the methodology 11

4.1 Tranz-Algo system architecture (as built) 15
4.2 UML-style class / entity diagram (MVP) 16
4.3 True Cost interaction flow 17
4.4 Dashboard — stock and shipment overview with Start fresh and Open True Cost 17
4.5 Products — catalogue entry form and CSV import 18
4.6 Shipments — create batch with dual cost mode (Already-Local / Auto FX) . . . 18
4.7 True Cost — Step 2 retail suggestion with margin slider and safety floor 19
4.8 Stock — available / low / out board with Record sale 19

List of Tables

2.1 Comparison of tool classes against the costing step Tranz-Algo performs 8

3.1 Tools and technologies 12

5.1 Implemented modules 22

6.1 Representative test cases and results 26
6.2 Demo A expected landed cost and suggested retail (ZMW) 27
6.3 Demo A landed cost when the same paperwork is allocated by weight (ZMW) . 27
6.4 Demo B landed cost by weight at rate 20 (ZMW) 28
6.5 Demo B landed cost by quantity at the same rate of 20 (ZMW) 28

8.1 Objective achievement for the MVP scope 34

D.1 Traceability from need to evidence 43

List Of Abbreviations And Acronyms

Abbreviation	Meaning
API	Application Programming Interface
CSV	Comma-Separated Values
ERP	Enterprise Resource Planning
FX	Foreign Exchange
JWT	JSON Web Token
MVP	Minimum Viable Product
POS	Point of Sale
SME	Small and Medium-sized Enterprise
UI	User Interface
VAT	Value Added Tax
ZMW	Zambian Kwacha

Note On Group Projects

This is an individual report for a two-member group project. The shared system is Tranz-Algo. The co-author is Musonda Katongo. Each student submits only their own individual report (not a joint Word document). The shared source code and documentation live in the project Git repository: `https://github.com/Nixon-Shibuchi/Tranz-Algo`. Individual responsibilities for Nixon Shibuchi are stated in Chapter 5 and Appendix A. Each student remains responsible for understanding the full system and for the honesty of their contribution claims.

Chapter 1

Introduction

1.1 Background and Context

Shops in Zambia that depend on imports rarely buy one item on its own. A shipment comes with a supplier invoice, a freight bill, and a clearing sheet where duty and VAT are written as totals for the whole consignment. So the price on the supplier invoice is only part of what the item costs by the time it is on the shelf. Transport and tax have to be shared across the units. The invoices also mix foreign currency with figures the shipper or the clearing agent has already put in kwacha. Whether the retail price is safe depends on how those shared costs are split.

Most owners do this split in a spreadsheet, or they put a rough markup on the supplier unit price and move on. A normal till system does not help there because it only records the sale after the price exists. A full accounts system can work out landed cost but it also wants purchasing and ledgers. What I was after is the smaller job in between. Take one shipment's papers, cost each item, and come out with a retail price that can be explained.

1.2 Problem Statement

Counting what is on the shelf is the easy part. Showing, line by line, how the retail price came out of the shipment papers is harder. Freight, duty and VAT sit on the batch, and a different split can change which items look profitable. The papers are often split too, a foreign invoice with local clearing receipts for the same shipment. If the rate moves after the price is set, a price that looked safe can stop covering the cost.

Spreadsheets will do the sums. They do not keep the method in a way that is easy to defend later, because a cell can be changed and nobody can see the old rule. A till system starts after the price is already chosen. A full ERP is more system than this one decision needs. What is missing is a repeatable way to go from one shipment's papers to a landed cost and a suggested

price, and to leave that calculation on the batch.

1.3 Project Aim

To design, implement, and evaluate a free-to-run web system that turns import shipment paperwork into per-item landed cost, suggested retail price, auditable Cost history, and batch-linked inventory for SME administrators.

1.4 Project Objectives

- Analyse import costing needs and freeze dual-mode formulas (Auto FX and Already-Local).
- Design a three-tier architecture, data model, and True Cost workflow with allocation comparison.
- Implement FastAPI services, React admin UI, PostgreSQL persistence, and FX integration.
- Evaluate correctness against spreadsheet oracles (tolerance ≤ 0.01 ZMW) using demo scenarios.
- Document technology deviations from the proposal and produce operator / demo guidance.

1.5 Research Questions

- How can dual entry modes accept mixed foreign/local import paperwork without forcing one workflow?
- How can allocation of shared shipping/duty/VAT be made explainable to an SME user?
- Can automated landed-cost and price suggestion match a spreadsheet oracle within ± 0.01 ZMW?

1.6 Scope of the Project

1.6.1 In Scope

- Admin authentication (JWT); product catalogue; shipment batches and lines.

- Dual modes, allocation (quantity/weight), True Cost studio, safety floor, Cost history.
- CSV import for demos; stock board; Record sale (stock-out only).
- Frankfurter FX with cache/manual; PostgreSQL primary and SQLite fallback.

1.6.2 Out of Scope

- Full POS checkout (payments, customers, receipts).
- Legal ZRA/customs duty assessment APIs.
- React Native companion app (deferred; web admin delivered).
- Multi-tenant SaaS billing; barcode camera as a core path.
- Public deployment of the API and web interface. Local execution is sufficient to demonstrate and check the costing behaviour.

1.7 Significance of the Project

A shop can point at the batch when someone asks why a price is what it is. For the degree, the part worth showing is not another stock list. It is that the price can be traced when the papers mix a foreign invoice with kwacha clearing totals. The same demo files can be run again.

1.8 Limitations

I did not run a multi-shop trial or a retailer survey. Duty and VAT are whatever totals are written on the papers. The system does not work out the tax and it does not replace a clearing agent. It trusts what the user types. Value allocation is in the costing code, but the shipment screen offers quantity and weight. For the Auto FX demo I pin the rate at 20, otherwise a live fetch can change the result between runs and the same papers would not give the same answer.

For the demonstration and the tests I started the API and the web pages on one machine. A public address would let another device open the system. It would not change the landed costs, the suggested prices or the rate stored on the batch, because those come from rules I already checked against the spreadsheet. I left hosting until the formulas, the two cost modes and the admin screens were done. The database is separate. If DATABASE_URL points at a hosted PostgreSQL instance the rows are not stuck on the laptop, even while the app itself is local. Another device still needs the API deployed and both sides have to use that address.

1.9 Organisation of the Report

Chapter 2 reads the sources against the grocery shipment. Chapter 3 is how the system was built. Chapters 4 and 5 are the design and the implementation. Chapter 6 checks the figures against spreadsheets. Chapter 7 says what that check shows. Chapter 8 concludes. The appendices are the contribution statement, a short install guide, the worked lines, and the traceability table.

Chapter 2

Literature Review

2.1 Introduction

This chapter stays with one shipment, the grocery batch in Chapter 6. Landed cost, the exchange rate on that shipment, and the tools a small shop already uses are all read against that batch. Where two sources give different unit costs, I use the kwacha figures from that batch.

The same shipment is where I stopped following the books. They go on to hedging, warehouse modules, and what a customer will pay. Tranz-Algo keeps the landed cost, the rate that was used, and the allocation rule.

2.2 Key Concepts and Theories

Landed cost is more than the supplier invoice. Product price, freight, duty and the other charges together make the unit cost (Mangan & Lalwani, 2016; Christopher, 2016; Chopra, 2019; Bonanni, 2011). The usual ways to share a shipment total are quantity, weight or value. Cost-plus is still how many small firms set a starting price: landed unit cost times one plus the margin (Nagle & Müller, 2018). If the firm does not hedge, a moving exchange rate can wipe that margin (Eun & Resnick, 2018; Zulu, 2025).

I separate a charge that belongs to one SKU from a charge that belongs to the batch. The supplier unit price is the first kind. Shipping, duty and VAT usually arrive as totals for the consignment. If those totals are ignored, or dumped on one product, the other items look cheaper than they are and the margin looks safer than it is. Allocation is the rule that turns a batch total into a share per line, by quantity, by weight, or by local value. I froze those rules so two people with the same papers get the same landed cost.

The other awkward part is when the currency gets converted. Some supplier invoices are still

in foreign currency. Some clearing bills are already in kwacha. If the system only understands one of those, the user does the other one in a spreadsheet and that is where the mistakes come back. The two cost modes are there so both kinds of paper can be entered. The allocation and the price suggestion stay the same afterwards.

The books I used each cover a piece of this, not the whole path. The price depends on the landed unit, and that unit changes with the allocation rule before any markup. Which fees are even on the papers depends on the trade term. Foreign figures depend on a rate that has to be written down. I did not find one source that puts those steps together for a small importer.

Christopher (2016), Mangan and Lalwani (2016) and Chopra (2019) are clear enough that pricing off the supplier invoice alone is a bad idea. On the grocery shipment in Chapter 6 the rice invoice is K95. Share the batch charges by quantity and the landed cost is K170. Share them by weight and it is K323.77. Both can be called landed cost. The books list quantity, weight and value. They do not tell a small shop which one to pick, and they do not store the rule after the price is set. A spreadsheet can show either K170 or K323.77. Without the rule saved, I would not be able to say later which figure was the one that got committed.

That is why I read Nagle and Müller (2018) next to the costing books, not after them. They treat cost-plus as a way to get a starting price. Thirty percent on the quantity result is K221. The same thirty percent on the weight result is K420.90. The margin did not cause the gap. It made the allocation choice bigger. A floor at 10 percent above landed cost only helps if the landed cost is the one that was intended. They go on to what a customer will pay and what a competitor charges. I stop before that. The margin can still be changed later. An allocation that was never written down cannot be rebuilt.

Incoterms 2020 (International Chamber of Commerce, 2019) are where a right-looking sum can still be wrong. On some terms freight or duty is already in the supplier price. On others those charges come later as their own kwacha totals. Add freight that was already in the price and it gets counted twice. Incoterms are contract terms, not a costing program, and the system is not supposed to guess the term. Tranz-Algo does not read the term and it does not reject a total the user types. It keeps the figures from the papers, and the basis that was chosen, in Cost history.

Eun and Resnick (2018) split currency risk into the exposure on a known foreign invoice, and the slower effect on how a firm competes. One shipment is the first kind. It is converted once, for that decision. Zulu (2025) is the Zambian point: the kwacha has moved enough to change local prices, so leaving the rate off the batch is not a small thing. I store the rate on the shipment when the cost is saved. I do not let a later fetch rewrite that batch. Once the price is committed, a new market rate must not replace the kwacha figure the price was based on.

Pressman and Maxim (2020) and Sommerville (2016) matter here because the outputs can be far apart. The same papers can suggest K221 or K420.90 for one rice line, and only the allocation method changed. A screen that looks fine is not proof. I checked the outputs against a

spreadsheet worked separately, and I required agreement within K0.01. Whether shops would take the tool up is another question. It does not tell me whether K170 or K323.77 is what the chosen rule should produce.

That is also how I read the tools in the next section. A till system only sees the goods after a price exists, so allocation never comes up. A basic stock system keeps a cost somebody typed. An ERP can do the textbook landed cost, including warehouse modules a small importer is not trying to run. A spreadsheet can do the sums and still lose the rate, the rule and the totals. What I built is a record of that sequence, tied to the batch.

2.3 Existing Systems / Related Work

The tools a small shop already has fall into a few classes. A till is built around the sale. A stock list is built around the count. A spreadsheet will accept any formula that is typed into it. Clearing software produces the freight and duty papers and then stops. An accounts suite such as Odoo can allocate landed cost, and it also expects the rest of the business to live in that suite.

A till records the barcode, the payment and the receipt, and it takes the item off the stock. That is what the counter needs. It does not decide how K5,000 of shipping and K1,000 of duty should be shared across rice, sugar, oil, tea and soap that arrived together. A stock application is better than a book for quantities and a reorder level. The unit cost there is still a number someone typed, not a figure that can be rebuilt from the shipment.

For a small importer the spreadsheet is the real alternative. Any formula can be written, which is also the weakness. It lives in a cell. Someone can change the basis and leave no record of the old one. A later reader cannot tell whether the rate was 20 or the market rate that morning. Clearing systems move the goods and issue the invoices. They do not hold the shop’s catalogue or suggest a selling price. Odoo and SAP-class packages can allocate freight. They also bring purchasing, accounts and warehouse modules that a two-person shop may not be ready to run. I kept Tranz-Algo narrower than that.

2.4 Comparative Analysis

Table 2.1: Comparison of tool classes against the costing step Tranz-Algo performs

System type	Import batch true cost?	Dual modes?	Explainable allocation?	Fit for small importer
POS / shop apps	Rarely	No	No	Selling-focused
Basic inventory	Weak / manual	Usually no	Usually no	Quantity-focused
Spreadsheets	Possible, fragile	Manual	Opaque	Common but error-prone
Clearing / shipping	Paperwork only	Partial	No retail pricing	Logistics partner
ERP (Odoo/SAP-class)	Often yes	Varies	Possible, heavy	Often too complex
Tranz-Algo	Yes (core)	Yes	Yes (comparator + ledger)	Built for this niche

2.5 Research / Knowledge Gap

If this is described as another stock-counting app, the point has been missed. Inventory software tracks how many units are left. A till system records the sale. An ERP can cost a shipment but it is built for a bigger business than a small importer is running. A spreadsheet does the math and then does not keep the method. What was needed, and was not in those tools, is one workflow that accepts a foreign unit price or a figure already in kwacha, splits freight, duty and VAT by a rule that can be named, suggests a retail price from the landed cost, and keeps the inputs so the price can be explained after commit. Dual modes, the allocation comparison and Cost history are that workflow. They do not assess customs and they do not take payment.

For a Zambian importer the gap shows up as mixed papers. Phone parts can arrive with a dollar invoice and a kwacha clearing sheet. A grocery consignment can be local all the way through. Either way the same answers are needed: landed cost per unit, a suggested price at a chosen margin, a warning if that price is under the safety floor, and a record of what was committed. Zulu (2025) is why the rate is stored on the batch instead of letting a live ticker rewrite yesterday's cost. Eun and Resnick (2018) call this transaction exposure. The kwacha value of a foreign invoice is only fixed when the rate used for that decision is written down. Incoterms 2020 (International Chamber of Commerce, 2019) are why the papers differ. Some terms already include freight or duty in the seller's price. Others send those charges later as separate local totals. The system does not replace the Incoterm. It takes the totals the documents actually show.

2.6 Conceptual Framework

The framework is the order of the work. The papers go in, the frozen costing rules are applied, and the result is written to stock and to Cost history. Pressman and Maxim (2020) and Sommerville (2016) both treat a calculation like this as something that has to be checked. I took that

literally. The costing is the critical part, so I compared it with a spreadsheet worked separately. A screen that looked plausible was not the check.

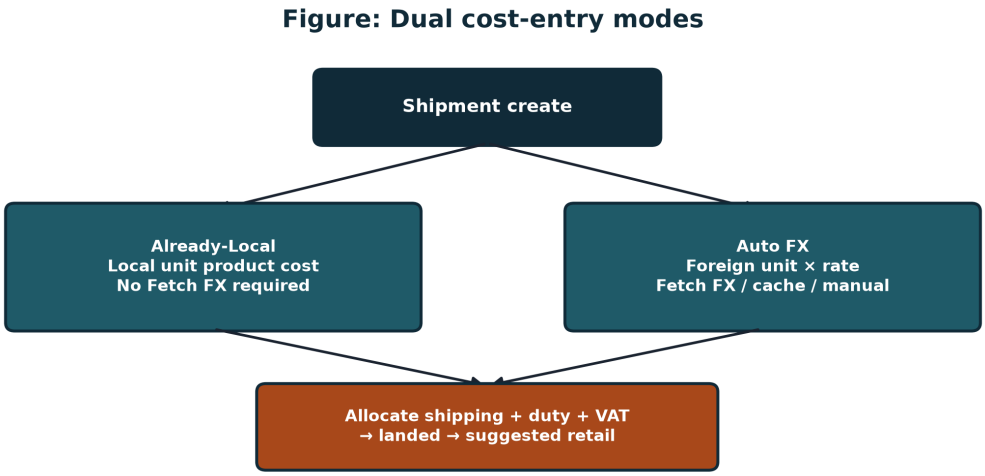


Figure 2.1: Dual cost-entry modes conceptual framework

2.7 Chapter Summary

Landed cost, if the books are taken seriously, is more than the figure on the supplier invoice, and the allocation rule changes the unit. On one line of rice the two results are K170 and K323.77. A cost-plus margin only scales the base that was chosen. It cannot fix a bad base. Incoterms decide which charges are on the papers, so the software should use the figures it is given and not invent fees. The rate belongs to that transaction so it stays with the batch.

One shipment can support two prices that both look defensible, so the screen was not trusted on its own. The costing was checked against a spreadsheet. Shops already have a till, a stock count, a full ERP, or a formula nobody can reconstruct. What was missing was one place that keeps the costing sequence.

Chapter 3

Methodology

3.1 Introduction

I kept the costing formulas fixed before I treated the build as finished software. After that I delivered the system in working slices, and I checked the figures against spreadsheets rather than against how the screen looked.

3.2 Research / Development Approach

The proposal named Waterfall for a reason I still agree with. The costing formulas had to be settled before I designed screens, because changing a form later does not repair a landed cost that was wrong. I kept that part. I did not keep a strict waterfall. That would have held every screen back until the design write-up was finished.

Once those formulas were checked, I built in slices: sign-in, products, shipments, True Cost, then stock. I showed each slice to my supervisor, Mrs. Leena Kumar. The CSV import came out of those meetings, so a demonstration does not mean typing every line. The formulas stayed as they were while the screens changed.

3.3 Requirements / Data Collection

The requirements I worked from were the approved proposal, design discussions with Musonda Katongo, and the two paperwork patterns we kept meeting: a foreign invoice, and clearing totals already in kwacha. The shipments I used for evaluation are made up, but the inputs are realistic. One is an Already-Local grocery batch. The other is Auto FX phone parts. The spreadsheet checks sit in docs/scenarios/.

I did not copy invoices from a trading business into the repository. Both demo batches show every input: quantities, weights, foreign or local unit prices, shipping, duty, VAT, margin, and, for Auto FX, the exchange rate. An examiner can recompute a line on a calculator, and the work does not depend on a shop's private records. What I can claim from that evaluation is that the formulas match and that the workflow can be operated. I cannot claim that every Zambian importer would use it.

3.4 System Architecture / Research Workflow

I used a three-tier layout: a React frontend, a FastAPI service under /api/v1, and PostgreSQL, with SQLite if no Postgres address is set. Exchange rates are fetched only when asked for. In True Cost the path is allocate, then suggest or adjust, then commit, then Cost history.

Figure: Tranz-Algo three-tier architecture

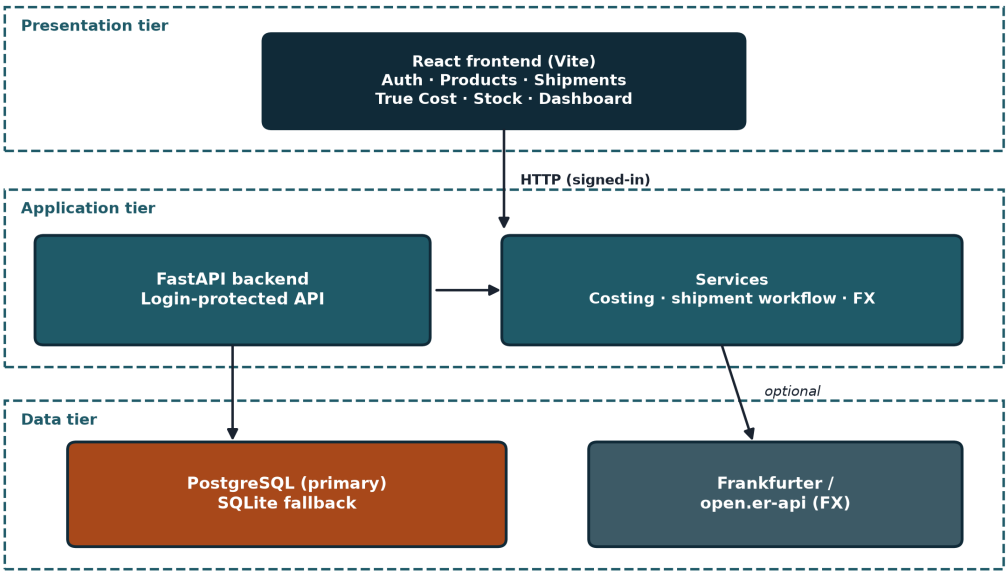


Figure 3.1: System architecture used in the methodology

3.5 Tools and Technologies

Table 3.1: Tools and technologies

Area	Choice
Languages	Python 3, JavaScript (React)
Backend	FastAPI, SQLAlchemy 2, Pydantic, JWT/bcrypt
Frontend	React (Vite), CSS themes
Database	PostgreSQL primary; SQLite fallback
FX	Frankfurter / open.er-api + cache/manual
Tests	pytest costing oracles
VCS	Git / GitHub
OS (dev)	Windows 10/11

3.6 Ethical and Legal Considerations

I hash passwords with bcrypt. I do not claim that the system assesses customs. The demo figures are made up. The free exchange-rate services are used on their published terms. What I built, and what my co-author built, is set out in Appendix A.

3.7 Evaluation Method

I counted a result as successful when three things held. The figures agreed with the spreadsheet within K0.01. The scenarios covered both cost modes, the allocation comparison, the safety floor, and the CSV path on the screen. Record sale reduced stock and did not turn into a point of sale.

3.8 Chapter Summary

I left the costing rules as they had been settled, built the three-tier system to the MVP we had scoped, and checked it against spreadsheets I can run again.

Chapter 4

System Analysis and Design

4.1 Introduction

This chapter is the design that was actually built: who uses the system, what it has to do, how the three tiers sit, and how one shipment becomes a price.

4.2 Stakeholder and User Analysis

- Primary user: import SME administrator (desk workflow).
- Indirect: shop staff who rely on defended retail prices and stock levels.
- Out of scope users: end customers at till; customs officers.

4.3 Functional Requirements

- FR1 Register/sign in with JWT.
- FR2 Manage products (including CSV import and optional auto SKU).
- FR3 Create shipments in Already-Local or Auto FX mode; enter lines and paperwork totals.
- FR4 Fetch/cache/manual FX for Auto FX.
- FR5 Allocate by quantity or weight; compare methods.
- FR6 Suggest retail prices; apply safety floor while adjusting; commit with Cost history.
- FR7 Update stock on commit; Record sale reduces stock only.

- FR8 Admin Start fresh / Clear drafts for demos.

4.4 Non-Functional Requirements

- NFR1 Correctness: money calculations within ±0.01 ZMW of oracle.
- NFR2 Security: bcrypt hashes; JWT on protected routes.
- NFR3 Usability: clear True Cost stages; Create account (not create admin).
- NFR4 Deployability: PostgreSQL via DATABASE_URL; SQLite fallback.
- NFR5 Maintainability: Strategy/Facade patterns for modes and commit.

4.5 Use Cases / User Stories

- As an admin, I import Demo A CSV and cost an Already-Local grocery batch.
- As an admin, I open Auto FX at rate 20 (CSV / UI starter), allocate by weight, and cost Demo B.
- As an admin, I review Cost history after commit and record a sale that reduces stock.

4.6 System / Solution Architecture

Figure 4.1 shows the as-built architecture. Three choices differ from the proposal. Each one follows from the costing task rather than from a change of aim.

The proposal named a React Native client. The person who costs a shipment works from an invoice and a clearing sheet, and needs the lines of a batch on one screen. The delivered client is the React web administration interface. A phone companion was deferred because the MVP had to finish the costing engine and those desk screens. It remains future work, not a dropped objective.

The proposal treated one hosted Supabase database as required. The build reads PostgreSQL from DATABASE_URL. The same code can use a local database or a hosted Postgres instance, including Supabase, when that URL is set. SQLite is only the fallback when no Postgres URL is present. The costing rules do not depend on one host, and a demonstration still keeps its data after the browser is closed.

The proposal described a continuously streaming exchange rate. A shipment is costed when its papers are in hand, and the rate used for that decision has to stay with the batch. A live ticker

would move the rate after a price had been committed. Fetch FX therefore runs when the user asks for it, the last successful rate is cached, and the user can type a rate. The rate stored on the shipment is the one used for that cost.

Figure: Tranz-Algo three-tier architecture

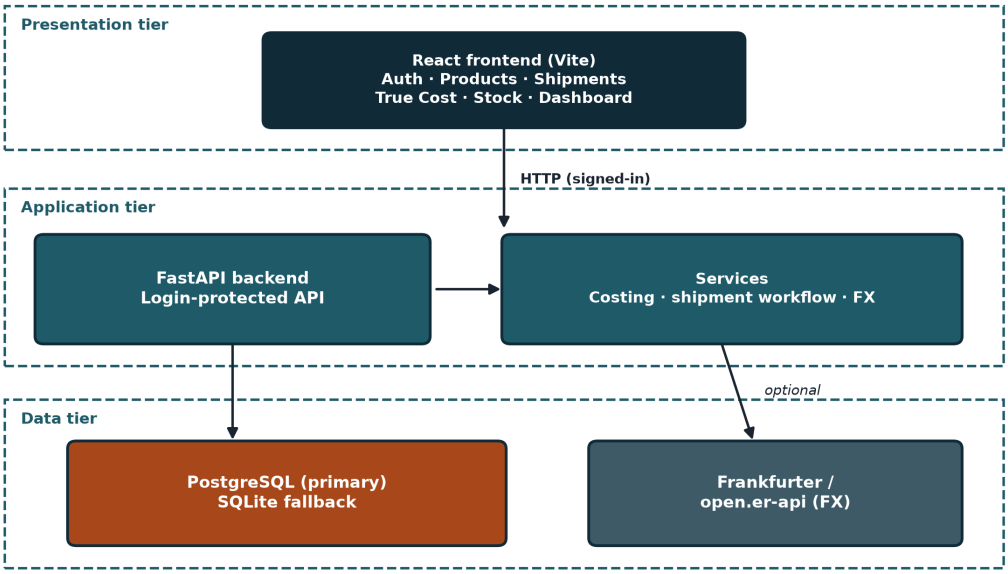


Figure 4.1: Tranz-Algo system architecture (as built)

4.7 Database / Data Design

Core entities: users, products, shipment_batches, batch_items, cost_entries, ledger_entries, sales, fx_rate_cache. Tax profiles were removed; duty/VAT live as batch paperwork totals. Primary keys are integer serials.

Figure: UML-style class / entity diagram (Tranz-Algo MVP)

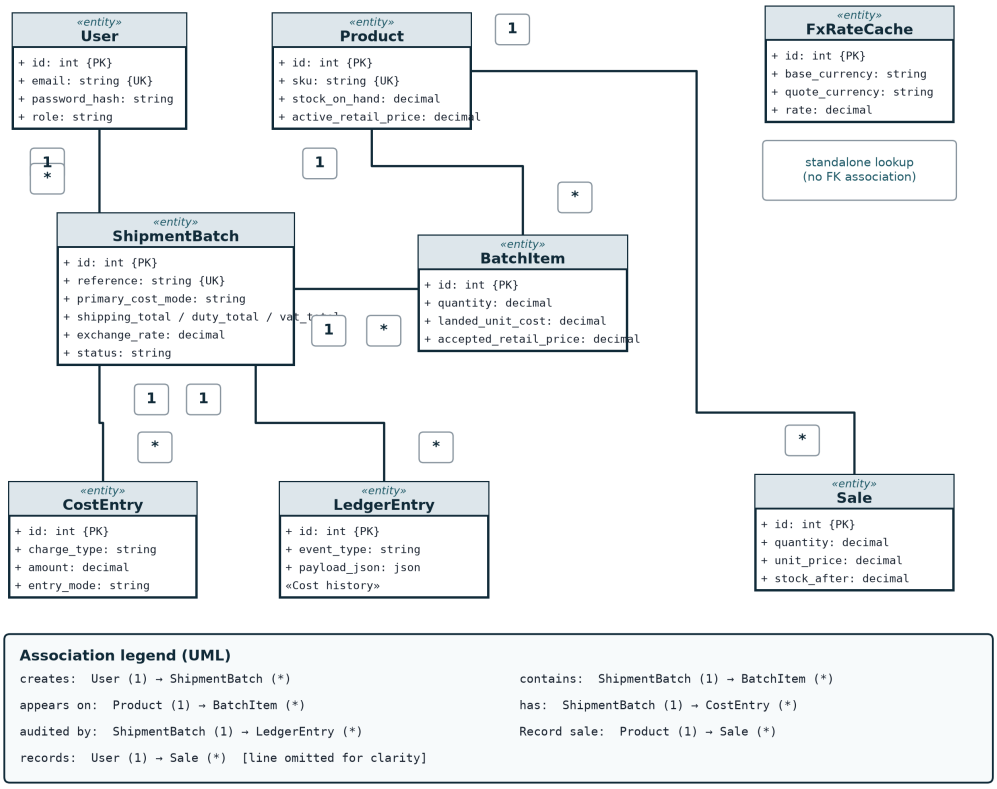


Figure 4.2: UML-style class / entity diagram (MVP)

4.8 Interface / Interaction Design

The UI is a React frontend with Welcome, Login/Create account, Dashboard, Products, Shipments (CSV import, Clear drafts), Shipment detail (Fetch FX), True Cost studio (Lines → Allocate → Price → Commit), Stock, and Cost history (When / What). Themes and a closable sidebar support desk use. Figures 4.4–4.8 are as-built screenshots of the working MVP.

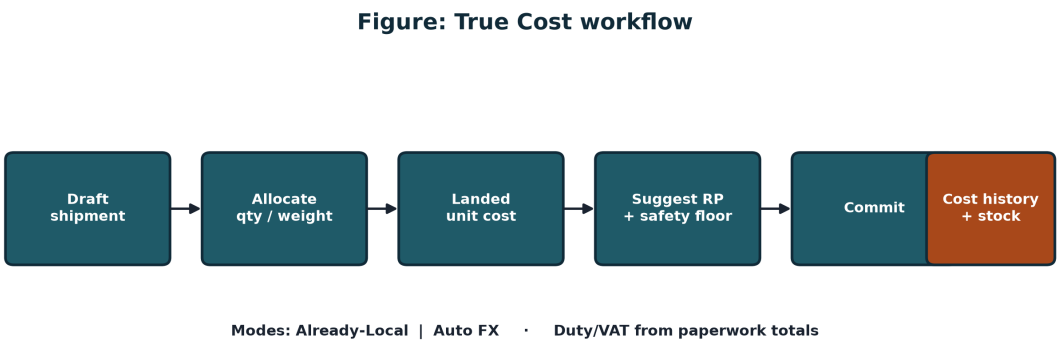


Figure 4.3: True Cost interaction flow

Figure 4.4 is the administrator's home screen after sign-in. It shows how many units are on hand, how many products are low or out of stock, and how many shipment batches are still drafts versus already committed. The short on-the-shelf list lets the user see retail prices without opening each product. Start fresh clears business data and keeps the user account, which is the safe reset before a demonstration. Open True Cost is the path into allocation and pricing. The screen therefore proves that stock and shipment status are visible before any costing decision is made.

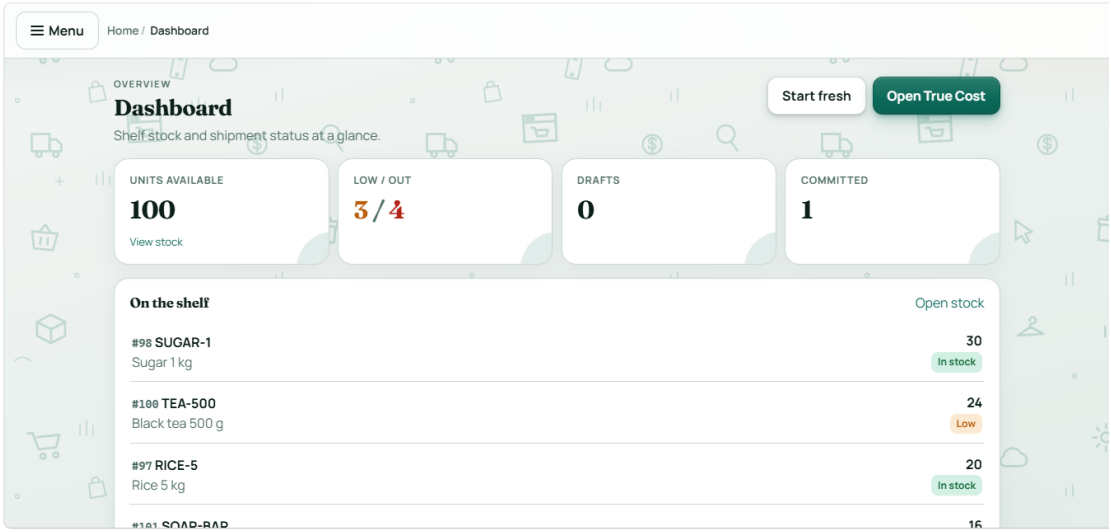


Figure 4.4: Dashboard — stock and shipment overview with Start fresh and Open True Cost

Figure 4.5 is the product catalogue. A product can be typed in by hand, and the SKU can be left blank so the system assigns one. Import CSV loads the demo catalogue in one step, including weight, unit, and reorder level. The reorder level is the low-stock threshold for that SKU only: stock is marked Low when the quantity on hand is above zero and at or below that level. Folded optional fields keep the form short. This screen is what makes later shipment lines possible, because a batch can only cost products that already exist.

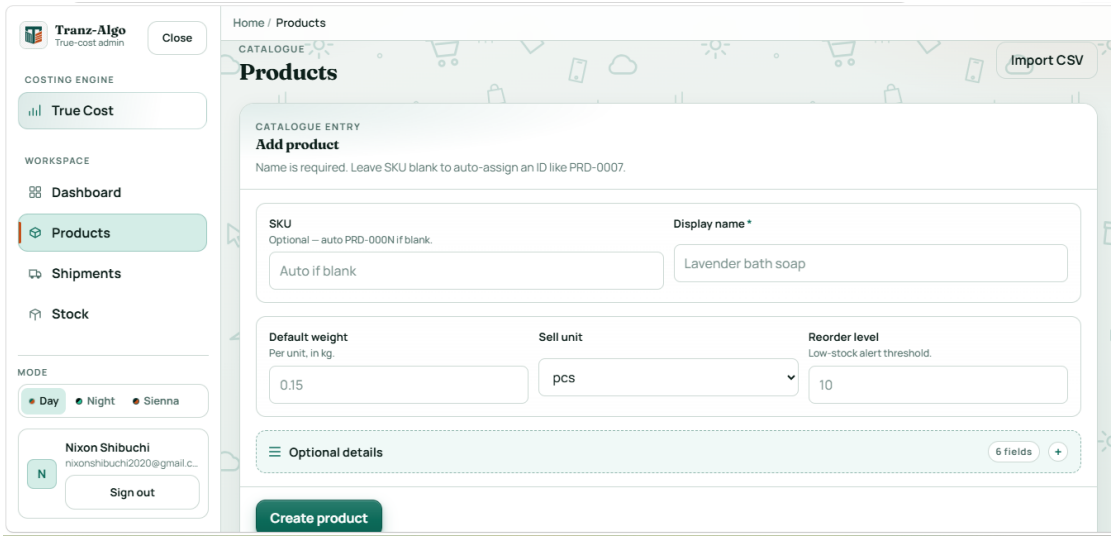


Figure 4.5: Products — catalogue entry form and CSV import

Figure 4.6 is where a shipment batch is created. The user chooses the cost mode once: Already-Local, when the paperwork is already in kwacha, or Auto FX, when product prices are still in foreign currency. Lines and a later CSV import follow that choice, so the mode is not re-decided on every row. Shipping, duty, and VAT are entered later as local paperwork totals in both modes. Clear drafts removes uncommitted batches, and Import shipment CSV loads Demo A or Demo B. The screen shows that mixed paperwork is handled by a deliberate mode, not by a hidden formula.

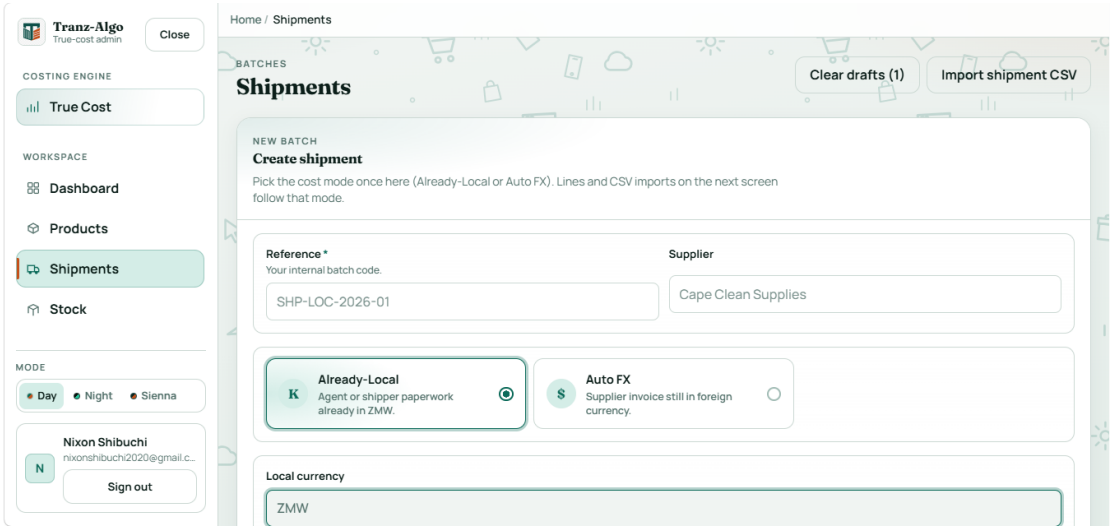


Figure 4.6: Shipments — create batch with dual cost mode (Already-Local / Auto FX)

Figure 4.7 is the pricing step of True Cost, after shared charges have been allocated. Dragging the margin updates every suggested retail price from landed unit cost times one plus the margin. The mark at 10 percent above landed cost is the safety floor. A manual price below that floor is warned before commit; the system does not block the save, and it does not recheck the floor

after the batch is committed. This screen is the visible link between the costing formulas and the price the shop will actually use.

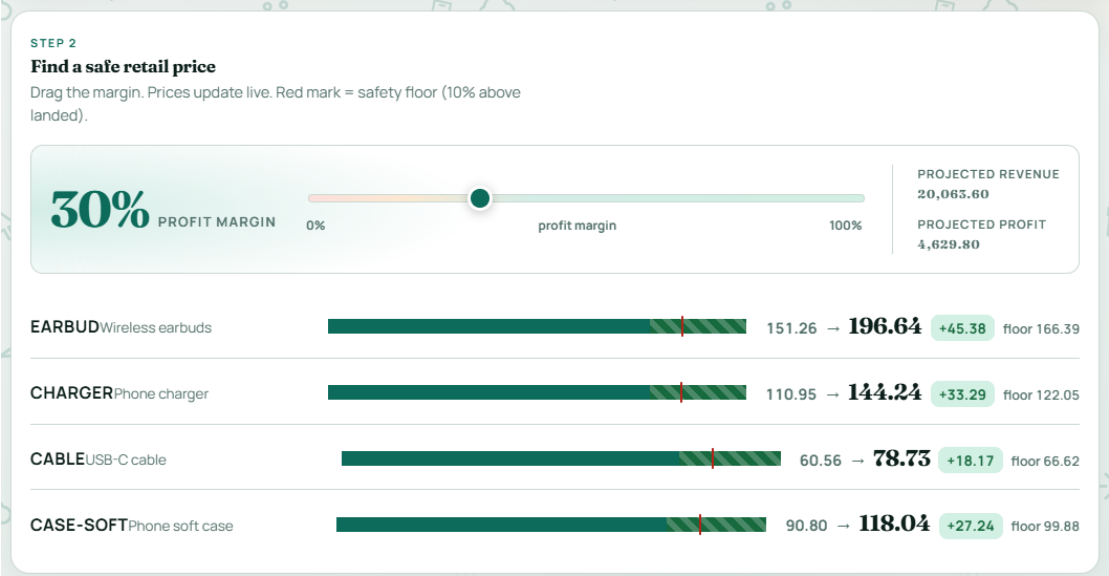


Figure 4.7: True Cost — Step 2 retail suggestion with margin slider and safety floor

Figure 4.8 is the stock board after a batch has been committed. Each SKU shows units on hand, whether it is in stock, low, or out, and the retail price accepted at commit. Record sale reduces quantity only. It does not take payment, print a receipt, or calculate change, so the screen is inventory truth rather than a point of sale. Low stock appears only when quantity is at or below that product's own reorder level, which is why the demo catalogue was set so a full Demo A and Demo B commit does not open as a wall of low-stock warnings.

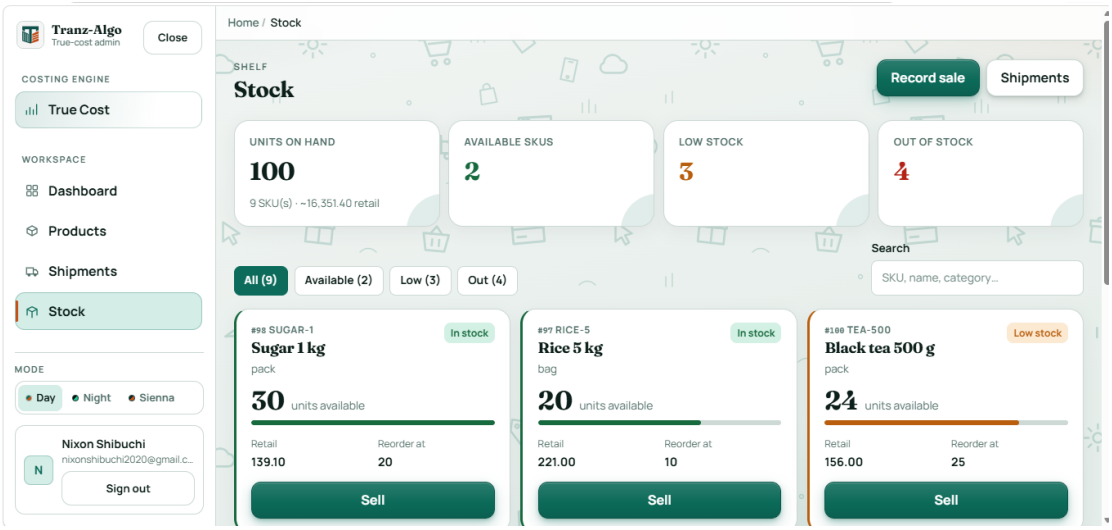


Figure 4.8: Stock — available / low / out board with Record sale

4.9 Algorithms / Models / Technical Design

Auto FX product local = foreign_unit_price × exchange_rate.

Already-Local product local = local_unit_product_cost.

Shared shipping/duty/VAT shares use quantity or weight proportions. Landed_unit = (product_local_line + shares) / qty. Suggested_RP = Landed_unit × (1 + margin). Safety floor = Landed_unit × (1.10) on live True Cost adjust (no post-commit recheck UI).

Design patterns: Strategy for cost modes/allocation; Facade for allocate/suggest/commit; Chain of Responsibility for FX providers.

Listing 4.1 is the costing function reduced to the lines that implement the formulas. local_unit_cost chooses Already-Local or foreign price times the exchange rate. allocate_shares splits each paperwork total by the selected basis (quantity or weight). Landed unit cost is that line total divided by quantity, and the suggested retail price applies the margin. The safety-floor check, in the same file, flags a price below landed unit cost times 1.10.

backend/app/services/costing.py (abridged)

```
unit = local_unit_cost(item, exchange_rate)
line_product = money(unit * qty)
basis = get_allocation_strategy(method).basis(prepared)
ship_shares = allocate_shares(basis, shipping_total)
duty_shares = allocate_shares(basis, batch_duty_total or 0)
vat_shares = allocate_shares(basis, batch_vat_total or 0)
line_landed = money(line_product + ship_share + duty + vat)
unit_landed = money(line_landed / quantity)
suggested = money(unit_landed * (1 + profit_margin))
```

Worked arithmetic is reported in full in Chapter 6 and Appendix C. The design rule used there is worth stating beside the formulas. For quantity allocation, each line's share of a total equals its quantity divided by the batch quantity, multiplied by that total. For Demo A the batch quantity is 100, so each unit carries K50 of shipping, K10 of duty, and K15 of VAT. Landed unit cost is the local product unit cost plus those K75. For weight allocation the same idea uses kilograms instead of units. The exchange rate is an input to Auto FX product cost only; it is not applied again to shipping, duty, or VAT, because those paperwork totals are already local in both modes.

FxRateCache is deliberately not a child of ShipmentBatch. The cache stores the last known rate for a currency pair so the next Auto FX batch can start from it. The shipment stores the rate actually used when its costs were saved. Updating the cache later must not rewrite a batch that has already been costed. Already-Local shipments do not require a rate at all, so a mandatory foreign-key from every shipment to an FX row would mis-state the domain.

4.10 Chapter Summary

Analysis and design freeze dual modes, three-tier architecture, ERD, and True Cost algorithms for implementation.

Chapter 5

Implementation / Development

5.1 Introduction

This chapter is the system as it runs. I cover the machine it was built on, the modules that do the costing, and which of that work was mine.

5.2 Development Environment

Development used Windows with Python virtualenv, Node/Vite for the frontend, PostgreSQL through DATABASE_URL (a local or Docker database, or a hosted instance), and Git. Backend runs on port 8000; frontend on 5173.

5.3 System Components / Modules

Table 5.1: Implemented modules

Module	Backend	Frontend
Auth	api/v1/auth.py	LoginPage / Create account
Products	api/v1/products.py	ProductsPage + CSV
Shipments	api/v1/shipments.py	Shipments / Detail + CSV
True Cost	shipment_workflow.py, costing.py	TrueCostPage
FX	fx.py, fx_providers.py	Fetch FX on costs
Stock	inventory.py	InventoryPage / Record sale
Admin	admin.py	Dashboard Start fresh

5.4 Key Implementation Details

PRIMARY_COST_MODES exposes Already-Local and Auto FX only. Duty/VAT are paperwork totals on the batch. Auto FX seeds rate from last cache or starter 20 until Fetch FX/manual. Commit writes ledger payloads for Cost history and increments stock. Record sale decrements stock without payments or receipts.

5.5 Integration

The React frontend calls JSON APIs with Bearer JWT. SQLAlchemy maps models to PostgreSQL/SQLite. FX providers are chained with fallback. CSV import parses demo samples into products and shipments.

5.6 Challenges and Solutions

Four problems kept coming up while I was building. Each one is handled in the code, not left as a note for later.

- Mixed paperwork. The shipment is created in Already-Local or Auto FX mode. Duty and VAT stay local paperwork totals in both modes, so the user is not asked to invent a third “mixed” mode.
- Desk workflow versus a mobile client. The admin work is data entry, allocation, and commit at a computer. The delivered client is a React web app. A phone companion was left as future work so the costing engine could be finished and tested.
- Exchange-rate service downtime. Fetch FX tries providers in order, then the last good cache, and the user can type a rate. Demo B is pinned at 20 so the written oracle and the screen show the same number.
- Repeated round trips when importing a CSV. Product and shipment imports use bulk endpoints so a demo file is one request, not one request per row. That matters on a hosted database where each call pays network latency.

Commit is where a draft stops being editable. The batch locks, the ledger stores what was allocated and which prices were accepted, stock increases by the line quantities, and the product takes the retail price from that commit. If the papers were wrong, the operator opens a new batch. The old one stays as it was. Start fresh clears the business data and leaves the user accounts, which is the reset I use before a demonstration.

5.7 Individual Contribution (group projects)

Musonda Katongo and I settled almost all of the architecture, technology, and formula choices together before coding. My part was the implementation: the FastAPI backend, the costing engine, most of the frontend wiring, the exchange-rate service, the evaluation scenarios and spreadsheet checks, the literature drafts, and most of the commits. His part is the complementary work in Appendix A. I do not claim his work as mine.

5.8 Chapter Summary

The MVP realises the designed dual-mode True Cost system on FastAPI + React + PostgreSQL.

Chapter 6

Testing, Results and Evaluation

6.1 Introduction

Evaluation verifies that Tranz-Algo's costing matches spreadsheet oracles and that demo UI paths work.

6.2 Testing Strategy

- Unit/engine tests: pytest fixtures for Demo A/B oracles.
- Scenario scripts 01–05: modes, comparator, safety floor, UI CSV path.
- Manual system demo: Start fresh → import CSV → True Cost → commit → Cost history → Record sale.

6.3 Test Cases and Results

Table 6.1: Representative test cases and results

ID	Test case	Expected	Status
TC-01	Already-Local Demo A qty oracles	Landed/RP within ±0.01	PASS
TC-02	Auto FX Demo B (rate 20)	Landed/RP within ±0.01	PASS
TC-03	Allocation compare Demo A	Qty vs weight deltas match arithmetic	PASS
TC-04	Safety floor on True Cost	RICE floor 187; warn if below	PASS
TC-05	CSV UI path (Scenario 05)	Same batches via import	PASS
TC-06	Record sale stock-out	Stock decreases; no POS artefacts	PASS

6.4 Experimental Results

Demo A (SHP-DEMO-LOCAL-ZMW) locks grocery landed costs such as 170 / 107 / 160 / 120 / 93 with 30% suggested retail. Demo B (SHP-DEMO-AUTO-FX) uses the pinned FX rate 20. Weight allocation locks landed costs 136.25 / 145 / 46.25 / 92.50 (earbud suggested retail 170.31). Quantity allocation of the same paperwork locks 150.33 / 110.33 / 60.33 / 90.33 (earbud suggested retail 187.91). CI fixtures in backend/tests/test_costing.py encode these oracles. The tables below are the numbers an examiner can recompute. Correctness in this report rests on these frozen checks, not on whether a particular live demonstration ran without interruption.

Demo A — Already-Local, quantity allocation, margin 30 percent.

Inputs: shipping K5,000, duty K1,000, VAT K1,500, total quantity 100. Per unit the shared charges are K50 + K10 + K15 = K75. Landed unit = local unit + 75. Suggested retail = landed × 1.30.

Table 6.2: Demo A expected landed cost and suggested retail (ZMW)

SKU	Qty	Local unit	Landed unit	Suggested retail
RICE-5	20	95.00	170.00	221.00
SUGAR-1	30	32.00	107.00	139.10
OIL-2L	10	85.00	160.00	208.00
TEA-500	24	45.00	120.00	156.00
SOAP-BAR	16	18.00	93.00	120.90

The same Already-Local paperwork is then allocated by weight. This is the comparison the True Cost screen can run before a price is committed. Line weight is the catalogue weight times quantity: rice 5 kg times 20 is 100 kg, sugar 30 kg, oil 20 kg, tea 12 kg, and soap 0.12 kg times 16 is 1.92 kg. The batch weighs 163.92 kg. Shipping, duty, and VAT stay K5,000, K1,000, and K1,500. A line's share is its kilograms divided by 163.92. Suggested retail is still landed unit cost times 1.30. Table 6.2 remains the committed demonstration. The next table shows what changes if weight is chosen instead.

Table 6.3: Demo A landed cost when the same paperwork is allocated by weight (ZMW)

SKU	kg	Local unit	Landed unit	Suggested retail
RICE-5	100	95.00	323.77	420.90
SUGAR-1	30	32.00	77.75	101.08
OIL-2L	20	85.00	176.51	229.46
TEA-500	12	45.00	67.88	88.24
SOAP-BAR	1.92	18.00	23.49	30.54

Demo B — Auto FX at rate 20, weight allocation, margin 25 percent.

Local product unit = foreign unit × 20. Line weights sum to 14 kg. Per kilogram the shared charges are K200 shipping + K50 duty + K75 VAT = K325. A line's share is its kilograms times K325, then divided by quantity to reach a unit add-on. Suggested retail = landed × 1.25.

Table 6.4: Demo B landed cost by weight at rate 20 (ZMW)

SKU	Qty	kg	USD unit	Local unit	Landed unit	Suggested retail
EARBUD	40	2.0	6.00	120.00	136.25	170.31
CHARGER	30	6.0	4.00	80.00	145.00	181.25
CABLE	40	2.0	1.50	30.00	46.25	57.81
CASE-SOFT	40	4.0	3.00	60.00	92.50	115.63

One line is enough to show the method. Earbuds: 6.00 × 20 = K120 local. Weight share of shipping, duty, and VAT is 2/14 of K4,550 = K650, or K16.25 per unit. Landed unit = 120 + 16.25 = 136.25. Suggested retail at 25 percent = 136.25 × 1.25 = 170.3125, stored as 170.31. The automated test allows at most K0.01 difference from these figures. Rice in Demo A is simpler: 95 + 75 = 170, and 170 × 1.30 = 221. Its safety floor at 10 percent is 170 × 1.10 = 187, so a manual retail price below 187 is warned before commit.

The same Auto FX paperwork is then allocated by quantity, still at rate 20 and still at a 25 percent margin. Total quantity is 40 + 30 + 40 + 40 = 150. Each unit carries an equal share of the K4,550 of shipping, duty, and VAT. Table 6.4 remains the weight demonstration. Table 6.5 is the quantity reading of the same lines. Earbuds move from 136.25 to 150.33, because 40 units is a larger fraction of the batch than 2 kg of 14 kg. The charger moves the other way, from 145.00 by weight to 110.33 by quantity, because it is heavy for the number of units.

Table 6.5: Demo B landed cost by quantity at the same rate of 20 (ZMW)

SKU	Qty	USD unit	Local unit	Landed unit	Suggested retail
EARBUD	40	6.00	120.00	150.33	187.91
CHARGER	30	4.00	80.00	110.33	137.91
CABLE	40	1.50	30.00	60.33	75.41
CASE-SOFT	40	3.00	60.00	90.33	112.91

6.5 User Evaluation

I did not run a retailer survey. In project meetings I showed the working application to my supervisor, Mrs. Leena Kumar, and she asked for practical changes. One of them was CSV files for products and shipments, so a demonstration would not depend on typing every line. Those files are the samples in Chapter 6 and Appendix B.

6.6 Analysis of Results

The spreadsheet checks show that the costing engine follows the formulas that were frozen. The two modes both run. The safety floor warns before commit. It does not check the floor again after commit.

Tables 6.2 and 6.3 are the same grocery papers under two rules. By quantity every unit carries K75 of the shared charges, so soap lands at 93 and rice at 170. By weight, using the catalogue weights, rice lands at 323.77 and soap at 23.49. The heavy rice takes the freight and the light soap mostly misses it. True Cost shows that shift before commit. Weight is not claimed as the legal basis. The basis is visible, and changing it changes which item looks profitable. Value allocation is in the engine. The screen offers quantity and weight.

With the rate held at 20, the phone-parts batch does the same thing. Weight gives earbuds 136.25 and the charger 145.00 (Table 6.4). Quantity gives 150.33 and 110.33 (Table 6.5). The rate did not move. Earbuds are light but there are many of them, so quantity loads them more. The charger is heavy for the number of pieces, so weight loads it more. The mode and the allocation rule are separate. Already-Local is not stuck to quantity, and Auto FX is not stuck to weight.

What these results do not show matters too. There was no multi-shop trial and no retailer survey. What can be shown is the operator path: import products, import a shipment, allocate, set a margin, commit, read Cost history, and record a sale that reduces stock. That is enough for an MVP correctness claim. It is not a claim that every import SME will adopt the tool. Chapter 7 takes that limit up.

6.7 Objective-by-Objective Evaluation

- Formulas/analysis: reflected in design docs and tests — achieved.
- Architecture/design: Figures 4.1–4.3 — achieved.
- Implementation: running MVP — achieved.
- Evaluation ≤0.01 ZMW — achieved for demos.
- Operator and demo documentation — achieved.

6.8 Individual Results

The spreadsheet checks and the core engine results are my implementation work. The design remains joint with Musonda Katongo.

6.9 Chapter Summary

On the demo shipments the landed costs and the suggested prices stay within K0.01 of the spreadsheet.

Chapter 7

Discussion

7.1 Introduction

The checks in Chapter 6 are narrow on purpose. This chapter says what they answer about the problem in Chapter 1, and what they leave open.

7.2 Interpretation of Findings

The engine matches a spreadsheet on the demo papers, and the batch keeps the rule that produced the price. The two cost modes are there because the papers do arrive in both forms: a foreign invoice, and a clearing sheet already in kwacha.

Demo A and Demo B were costed on a spreadsheet and then again in the application. They agree to within one ngwee, K0.01, for quantity, for weight, for Already-Local, and for Auto FX at a stated rate. The shipment keeps its own exchange rate. A later Fetch FX can update the saved market rate in FxRateCache and does not rewrite a batch that was costed at 20. That is why Cost history is usable. The date of the commit and the figures on it stay put.

The edges of the system stayed where I had put them. Record sale reduces stock. It does not take payment, print a receipt, or work out change. Duty and VAT are shares of totals the user typed. Someone marking the report can take one rice line, or one earbud line, from Appendix C and recompute it. The check does not depend on an accounts module that cannot be opened.

7.3 Comparison with Existing Work

A till and a basic stock list both start after a retail price already exists. An ERP can cost the shipment, and then it asks the shop to run purchasing and a warehouse as well. A spreadsheet

can produce either of the two prices and then lose the method. Tranz-Algo stores the method with the batch.

7.4 Contributions of the Project

- Working dual-mode True Cost MVP with comparator and ledger.
- Justified technology refinements (web admin; Postgres + SQLite fallback; on-demand FX).
- Reproducible evaluation artefacts and operator documentation.

7.5 Limitations and Implications

The system treats the paperwork the user types as trusted. It does the sums. It does not assess duty legally. For a demo the live rate is pinned, otherwise the same papers can give a different answer. A wider trial with working shops would say more about whether people will use it. That trial was not done.

The other limit is hosting, not the arithmetic. The API and the web pages were run on one machine, not on a public address. A public server would let another device connect. It would not change the landed costs already checked against the spreadsheet. DATABASE_URL can still point at hosted PostgreSQL while the application itself runs locally.

7.6 Chapter Summary

The findings match the reading in Chapter 2. Import costing with mixed papers can be explained if the allocation rule is stored with the batch and then checked.

Chapter 8

Conclusion and Recommendations

8.1 Introduction

The build meets the objectives in Section 1.4 inside the boundary set in Section 1.6. The rest of this chapter is that result, what I would leave as it is, and what can still be added without reopening the formulas.

8.2 Summary of the Project

Tranz-Algo takes one shipment’s papers and returns a landed cost per item, a suggested retail price, a Cost history row, and a stock update. The two cost modes, the quantity and weight comparison, and the safety floor are there because the papers are mixed and the rate can move after a price has been set.

8.3 Conclusions

A small web system can support that pricing decision and still stay out of the till and out of a full accounts package. On the demo shipments the application agrees with the spreadsheet.

8.4 Achievement of Objectives

Table 8.1 ties each objective in Section 1.4 to something that can be opened: a figure, a table, or a test.

Table 8.1: Objective achievement for the MVP scope

Objective	Evidence in this report	Status
Analyse costing needs and freeze dual-mode formulas	Chapter 4 formulas; Appendix C	Met
Design a three-tier system and data model	Figures 3.1 and 4.1–4.2; Section 4.7	Met
Implement admin UI, API, persistence, and FX	Chapter 5; running MVP on PostgreSQL	Met
Evaluate against spreadsheet checks within K0.01	Tables 6.1 to 6.5	Met
Document operation and demo data	Appendix B; docs/scenarios samples	Met

The objectives that were not adopted remain out of scope on purpose: legal customs assessment, a full point of sale, payment gateways, and a mobile client. Treating them as unmet MVP objectives would misread the project boundary stated in Section 1.6.

8.5 Recommendations

- Keep demo rates pinned when presenting Auto FX publicly.
- Train users that duty/VAT are paperwork totals, not legal assessments.
- Use Cost history in disputes about how a retail price was derived.

8.6 Future Work

Later work does not require a new costing design. The cost modes and the allocation rules are separate strategies, so another split, such as a fuller value-based screen, can be added beside quantity and weight. The exchange-rate sources are a chain, so another provider can be added without changing the shipment calculation. Allocate, suggest, and commit stay one workflow. A further client, whether a deployed copy or a phone companion, calls the same API. The formulas already checked in Chapter 6 stay the fixed part.

- A deployed API and web interface, so an off-campus demonstration does not depend on one laptop.
- Optional React Native companion for costing away from a desk.

- Richer value-based allocation UI; scheduled margin alerts.
- One-click demo seed and structured SME usability study.

References

[1] Bonanni, A. (2011). Total landed cost for tactical supply chain decisions (Master's thesis). Politecnico di Milano. https://www.politesi.polimi.it/

[2] Chopra, S. (2019). Supply chain management: Strategy, planning, and operation (7th ed.). Pearson.

[3] Christopher, M. (2016). Logistics and supply chain management (5th ed.). Pearson.

[4] Eun, C. S., & Resnick, B. G. (2018). International financial management (8th ed.). McGraw-Hill.

[5] International Chamber of Commerce. (2019). Incoterms 2020: ICC rules for the use of domestic and international trade terms (ICC Publication No. 723E). ICC.

[6] Mangan, J., & Lalwani, C. (2016). Global logistics and supply chain management. John Wiley & Sons.

[7] Nagle, T. T., & Müller, G. (2018). The strategy and tactics of pricing: A guide to growing more profitably (6th ed.). Routledge.

[8] Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill.

[9] Sommerville, I. (2016). Software engineering (10th ed.). Pearson.

[10] Zulu, T. (2025). Exploring the influence of kwacha fluctuations on medium, small and micro enterprises (MSMEs) profitability: A case of Choma District, Southern Zambia. International Journal of Research and Innovation in Social Science, 9(11), 3427–3460. https://doi.org/10.47772/ijriss.2025.91100274

Appendix A

Individual Contribution Statement

Purpose: identify work for which Nixon Shibuchi was primarily responsible, consistent with the main report, and acknowledge the co-author's complementary work.

A.1 Project and Group Information

Item	Details
Project Title	Tranz-Algo: Explainable Landed-Cost Inventory Orchestration with Dual Cost Modes for Import-Dependent SMEs
Student Name	Nixon Shibuchi
Computer Number	2021437361
Group Size	2
Co-author	Musonda Katongo (2020583969)
Supervisor	Mrs. Leena Kumar
Source repository	`https://github.com/Nixon-Shibuchi/Tranz-Algo`
Submission form	Individual report only (shared code via Git URL)

A.2 Overall Group Project

The group developed Tranz-Algo, an explainable landed-cost inventory orchestration system with dual cost modes, True Cost studio, Cost history, and batch-linked stock for import SMEs.

A.3 Individual Role and Responsibilities

Primary role: implementation lead.

Primary implementation: FastAPI backend; costing engine and dual-mode formulas; most frontend integration; FX service; scenarios/oracles; literature lead drafts; majority of git commits (including pending updates).

A.4 Collaboration and Co-author Work

Joint (both): problem definition; architecture; technology selection and proposal deltas; ERD direction; dual modes and formulas; evaluation approach; presentation narrative.

Co-author (Musonda Katongo): Co-design partner for architecture, technology, and formulas before coding; literature review on related-work drafts; UI redesign and True Cost workflow implementation commit; viva/demo drive role for the live walkthrough.

Estimated contribution (honest): design ownership roughly shared; implementation effort majority Nixon Shibuchi, with substantive UI/True Cost contribution from Musonda Katongo. Neither member claims the other's exclusive code as sole personal work.

Supervisor verification: _____ Date: _____

Appendix B

User Manual / Installation Guide (summary)

Full source, README, and demo CSVs are in the Git repository: `https://github.com/Nixon-Shibuchi/Tranz-Algo`. Operator and demo guidance is summarised below; CSV samples live under docs/scenarios/samples/.

- Clone: git clone `https://github.com/Nixon-Shibuchi/Tranz-Algo.git`
- Backend: create venv, pip install -r requirements.txt, set DATABASE_URL, uvicorn app.main:app –reload
- Frontend: npm install && npm run dev
- Demo: Import products_demo.csv, then shipment_A_full.csv / shipment_B_full.csv from docs/scenarios/samples/
- True Cost → allocate → price → commit → Cost history; optional Record sale

B.1 Demonstration sequence

The shortest defensible demonstration is Demo A, because it needs no exchange-rate service. Import products_demo.csv, then shipment_A_full.csv (reference SHP-DEMO-LOCAL-ZMW). Open True Cost, allocate by quantity, set margin 0.30, and commit. Rice should land at K170 and suggest K221. If time remains, import shipment_B_full.csv (SHP-DEMO-AUTO-FX), keep the rate at 20, allocate by weight, and set margin 0.25. Earbuds should land at K136.25. Reorder levels in the product CSV sit below those committed quantities, so the stock board shows In stock rather than Low after both commits.

B.2 Persistence

The API reads DATABASE_URL. The project runs on PostgreSQL, including a hosted Postgres connection when that URL points at a cloud instance, with SQLite only as a local fallback if no Postgres URL is set. Tables are created when the API starts. Business data therefore survives a browser refresh. A second device sees the same data only when it uses the same deployed API and the same database URL; localhost on one laptop is not automatically visible to another machine.

Appendix C

Worked Costing Examples

This appendix repeats the arithmetic behind Tables 6.2 to 6.5 so a reader can check the engine without opening the source repository. Money is in Zambian kwacha unless a unit price is marked USD. Rounding for display is two decimal places.

C.1 Shared-charge identity

Let T be a paperwork total (shipping, duty, or VAT) and let b_i be the basis of line i (quantity or weight). The line share is (b_i / sum of b) × T. Unit add-on is that share divided by quantity. Landed unit cost is local product unit cost plus the three unit add-ons. Suggested retail is landed unit cost × (1 + margin). The safety floor used by the True Cost screen is landed unit cost × 1.10.

C.2 Demo A, rice line

Batch quantity is 20+30+10+24+16 = 100. Rice quantity is 20, so rice receives 20 percent of each total: shipping 1,000, duty 200, VAT 300. Per unit that is 50 + 10 + 15 = 75. Local unit cost is 95. Landed unit cost is 170. At a 30 percent margin the suggested retail is 221. The same K75 is added to sugar (32 → 107), oil (85 → 160), tea (45 → 120), and soap (18 → 93). Suggested retails at 30 percent are 139.10, 208.00, 156.00, and 120.90.

C.3 Demo A by weight, rice line

Line weight is catalogue weight times quantity. Rice is 5 times 20, which is 100 kg, and the batch is 100 + 30 + 20 + 12 + 1.92 = 163.92 kg. Rice therefore receives 100/163.92 of each

paperwork total. Shipping is 5,000 times that fraction, 3,050.27. Duty is 610.05. VAT is 915.08. The product line is 95 times 20, which is 1,900. Landed line is 1,900 + 3,050.27 + 610.05 + 915.08 = 6,475.40. Landed unit is 6,475.40 / 20 = 323.77. Suggested retail at 30 percent is 323.77 times 1.30 = 420.90. Soap, at 1.92 kg, lands at 23.49 rather than the quantity result of 93.00. A disagreement of more than K0.01 from Table 6.3 would mean the weight basis, not the Already-Local product prices, was wrong.

C.4 Demo B, earbud line at rate 20

Foreign unit price 6.00 USD × 20 = 120 kwacha. Line weights are 2 + 6 + 2 + 4 = 14 kg. Earbuds weigh 2 kg, so they receive 2/14 of shipping 2,800 (400), of duty 700 (100), and of VAT 1,050 (150). The line add-on is 650. Divided by quantity 40, the unit add-on is 16.25. Landed unit cost is 136.25. At a 25 percent margin the suggested retail is 170.31. The charger line is heavier (6 kg) and therefore carries a larger share, which is why its landed unit (145.00) exceeds the earbuds even though its local product unit (80) is lower.

C.5 Demo B by quantity, earbud line at rate 20

Total quantity is 40 + 30 + 40 + 40 = 150. The local unit is 6.00 USD times 20, which is 120. Earbuds are 40/150 of each paperwork total. Shipping is 2,800 times that fraction, 746.67. Duty is 186.67. VAT is 280.00. The product line is 120 times 40, which is 4,800. Landed line is 4,800 + 746.67 + 186.67 + 280.00 = 6,013.34. Landed unit is 6,013.34 / 40 = 150.33. Suggested retail at 25 percent is 150.33 times 1.25 = 187.91. The same earbuds land at 136.25 when the basis is weight, because 40 units is a larger share of 150 units than 2 kg is of 14 kg. The charger, heavy for its quantity, moves from 145.00 by weight to 110.33 by quantity. Any remainder of a ngwee is added to the last line so the shares still sum to the paperwork totals. A disagreement of more than K0.01 from Table 6.5 would mean the quantity basis, not the rate of 20, was wrong.

C.6 What a mismatch would mean

If the application disagreed with these figures by more than K0.01, the defect would be in the costing service or in a wrong rate or allocation basis, not in the user interface. The automated tests lock the expected values for that reason. A live screen that shows a different rate than 20 on Demo B is not a formula failure; it is a different input, and the oracle is defined only at rate 20.

Appendix D

Requirements Traceability

The table traces the main functional needs from analysis to the module that implements them and the check that demonstrates them.

Table D.1: Traceability from need to evidence

Need	Design	Implementation	Check
Sign in before costing	Login-protected API	auth.py, JWT	Unauthenticated calls rejected
Catalogue of import SKUs	Product entity	products API + page	products_demo.csv import
Already-Local paperwork	Line-cost strategy	costing.py	Table 6.2 / TC-01
Auto FX paperwork	Foreign × rate	fx.py + line strategy	Tables 6.4 and 6.5 / TC-02
Split shipping, duty, VAT	Qty or weight strategy	strategies/	TC-03 comparator
Suggest and guard price	Margin and floor	True Cost page	TC-04 rice floor 187
Audit after commit	Ledger entries	Cost history	Commit locks the batch
Stock after receipt	stock_on_hand	inventory API	TC-06 record sale
Recover from FX outage	FxRateCache	fx service chain	Cache then manual rate

Low stock is not a global constant. A product is Low when stock on hand is greater than zero and less than or equal to that product's reorder level. It is Out at zero. The demo catalogue sets reorder levels below the quantities committed by Demo A and Demo B, so a full sample load does not open on a wall of low-stock warnings.